

RentLoop – Sistem preporuke (Recommender System)

RentLoop je platforma za iznajmljivanje nekretnina koja korisnicima omogućava pretraživanje i rezervaciju smještaja na osnovu lokacije, cijene, tipa najma i drugih kriterija. Kako broj oglasa (listing-a) i korisnika raste, korisniku postaje teže ručno pronaći najrelevantnije opcije. Zbog toga je unutar sistema implementiran recommender mehanizam koji personalizuje sadržaj i korisniku prikazuje preporučene listing-e kroz endpoint **/recommended**, a kada nema dovoljno ili nema nikakvih personalizacijskih rezultata, sistem može preći na fallback logiku kroz **/popular**.

U RentLoop aplikaciji implementirane su dvije povezane funkcionalnosti:

1. **Personalizovana preporuka listing-a (/recommended)**
2. **Popularni listing-i / fallback preporuke (/popular)**

Objekti funkcionalnosti su implementirane kao backend endpointi u okviru **ListingsController** i zasnovane su na **rule-based hibridnom rangiranju**.

1) Personalizovana preporuka listing-a (/recommended)

Opis pristupa

Personalizovane preporuke se generišu na osnovu kombinacije sljedećih signala:

- korisnikove historije pregleda listing-a
- korisnikove historije rezervacija
- globalne popularnosti listing-a u sistemu
- kvaliteta listing-a kroz ocjene i broj recenzija

Sistem formira skup kandidata iz aktivnih listing-a, zatim svakom kandidatu dodjeljuje numerički **Score**, te korisniku vraća najbolje rangirane rezultate.

1.1. Prikupljanje korisničkih signala

Za autentifikovanog korisnika sistem prikuplja sljedeće podatke:

A) Zadnjih 10 pregledanih listing-a (ListingViews)

- uzima se do **10** listing-a koje je korisnik posljednje gledao

- sortiranje se vrši po ViewedAt opadajuće
- koristi se Distinct() kako bi se izbjeglo ponavljanje istog listing-a

Ovaj dio predstavlja signal stvarnog interesa korisnika — šta je korisnik nedavno pregledao.

B) Zadnjih 5 rezervisanih listing-a (Reservations)

- uzima se do 5 listing-a koje je korisnik rezervisao
- sortiranje se vrši po CreatedAt opadajuće
- koristi se Distinct() da se izbjegnu duplikati

Ovaj dio predstavlja najjači signal stvarne korisničke preferencije, jer rezervacija označava konkretnu akciju, a ne samo pregled.

C) Exclude lista

Listing-i koje je korisnik već rezervisao stavljaju se u **exclude listu** i izbacuju iz kandidata za preporuku.

Na taj način sistem ne preporučuje korisniku iste listing-e koje je već rezervisao.

1.2. Globalni signal popularnosti

Pored korisničke historije, sistem koristi i agregirane podatke o popularnosti listing-a u cijeloj aplikaciji.

Pripremaju se dvije mape:

A) Pregledi u zadnjih 7 dana

Za svaki listing se broji koliko je puta pregledan u posljednjih 7 dana.

Rezultat se sprema u mapu:

- `views7d[ListingId]` = broj pregleda

B) Rezervacije u zadnjih 30 dana

Za svaki listing se broji koliko puta je rezervisan u posljednjih 30 dana.

Rezultat se sprema u mapu:

- `res30d[ListingId]` = broj rezervacija

Ove dvije mape služe kao globalni signal popularnosti i koriste se pri izračunu konačnog skora.

1.3. Formiranje historijskog profila korisnika

Nakon što se uzmu ID-evi iz zadnjih pregleda i zadnjih rezervacija, oni se spajaju u jedinstvenu listu:

- `historyIds = recentViewedIds + recentReservedIds`

Zatim se iz baze učitavaju osnovni profili tih listing-a. Za svaki historijski listing koriste se sljedeća polja:

- `Id`
- `CityId`
- `RentTypeId`
- `PricePerNight`
- `RoomsCount`

Ovi podaci se mapiraju u pomoćni profil `ListingHistoryProfile`.

Svrha ovog koraka je da sistem dobije referentni profil korisnikovih prethodnih interesa i ponašanja, kako bi mogao procijeniti sličnost novih kandidata sa onim što je korisnik ranije gledao ili rezervisao.

1.4. Kandidati za preporuku

Baza kandidata se formira iz entiteta `Listings` prema sljedećim pravilima:

- uzimaju se samo listing-i gdje je `IsActive == true`
- iz kandidata se izbacuju listing-i koje je korisnik već rezervisao
- kandidati se sortiraju po `CreatedAt` opadajuće
- uzima se **najviše 250 najnovijih listing-a**

Drugim riječima, recommender ne razmatra cijelu bazu, nego ograničen skup novijih i aktivnih listing-a.

Za svakog kandidata dodatno se učitavaju i/ili izračunavaju sljedeći podaci:

- osnovni podaci listing-a:
 - `Id`
 - `Name`
 - `PricePerNight`

- City
- RentType
- CityId
- RentTypeId
- RoomsCount
- MaxGuests
- DistanceToCenterKm
- CreatedAt
- vizuelni prikaz:
 - CoverUrl

Sistem prvo pokušava pronaći sliku označenu kao cover (`IsCover == true`), a ako ona ne postoji, uzima prvu sliku po `SortOrder`.
- kvalitet listing-a:
 - AvgRating = prosječna ocjena iz Reviews
 - ReviewsCount = ukupan broj recenzija

Na taj način kandidat sadrži i funkcionalne i kvalitetne informacije potrebne za rangiranje i prikaz korisniku.

1.5. Izračun ukupnog Score-a

Za svakog kandidata računa se integer vrijednost Score. Score se sastoji iz tri glavne grupe signala:

A) Sličnost sa korisničkom historijom (History similarity)

Za svaki listing iz `historyListings` kandidat dobija bodove ako mu je sličan.

Pravila su:

- ako kandidat ima isti `CityId` kao historijski listing → +6
- ako kandidat ima isti `RentTypeId` → +5
- ako je cijena kandidata u opsegu $\pm 20\%$ u odnosu na historijski listing → +4
- ako kandidat ima isti `RoomsCount` → +2

Važno je da se ova logika izvršava kroz petlju nad više historijskih listing-a. To znači da kandidat može akumulirati više bodova ako odgovara većem broju elemenata iz korisnikove historije.

Na primjer, listing koji je sličan većini prethodno pregledanih ili rezervisanih nekretnina dobit će znatno veći score od listing-a koji ne liči na korisnikove prethodne izbore.

B) Popularnost (Popularity signal)

Kandidat dobija dodatne bodove prema globalnoj aktivnosti drugih korisnika u sistemu.

Pregledi u 7 dana

Ako listing postoji u mapi views7d, dobija:

- $+ \min(10, \text{viewsCount} / 3)$

Dakle:

- broj pregleda se dijeli sa 3
- maksimalni doprinos ovom dijelu score-a je **10**

Rezervacije u 30 dana

Ako listing postoji u mapi res30d, dobija:

- $+ \min(15, \text{reservationsCount} * 2)$

Dakle:

- svaka rezervacija daje jači signal od pregleda
- maksimalni doprinos ovom dijelu score-a je **15**

Ovim se osigurava da popularni listing-i dobiju dodatni boost, ali bez potpunog dominiranja nad ostalim signalima.

C) Kvalitet listing-a (Quality signal)

Pored sličnosti i popularnosti, u score ulazi i kvalitet sadržaja kroz ocjene i recenzije.

Prosječna ocjena

Kandidat dobija:

- $+ \min(10, \text{AvgRating} * 2)$

Na primjer:

- rating 4.5 daje približno 9 bodova
- maksimalni doprinos je **10**

Broj recenzija

Kandidat dobija:

- $+ \min(5, \text{ReviewsCount} / 10)$

Dakle:

- svakih 10 recenzija daje dodatni 1 bod
- maksimalni doprinos je **5**

Ovaj dio pomaže da bolje ocijenjeni i više potvrđeni listing-i budu rangirani više.

1.6. Rangiranje i prikaz rezultata

Nakon što se za svakog kandidata izračuna score, kandidati se transformišu u izlazni format i sortiraju po sljedećim pravilima:

1. prvo po Score opadajuće
2. zatim po CreatedAt opadajuće

To znači da kod jednakog score-a prednost imaju noviji listing-i.

Na kraju sistem vraća:

- take rezultata
 - podrazumijevana vrijednost je **15**
 - dozvoljeni raspon je od **1 do 50**
-

1.7. Fallback logika

Ako nakon rangiranja nema rezultata, sistem automatski aktivira fallback:

- `return await Popular(take);`

To znači da se u slučaju prazne liste personalizovanih preporuka korisniku ipak prikazuju relevantni listing-i, ali sada prema globalnoj popularnosti umjesto personalizacije.

Važno je naglasiti da u trenutnoj implementaciji fallback nije eksplicitno vezan za provjeru “da li korisnik ima dovoljno signala”, nego se aktivira kada rezultat preporuke ostane prazan.

1.8. Cold Start napomena

Za nove korisnike ili korisnike sa vrlo malo interakcija, personalizacija je ograničena jer sistem nema dovoljno historijskih podataka.

U toj situaciji:

- historijski dio scoring-a praktično ne doprinosi rezultatu
- veći značaj preuzimaju popularnost i kvalitet listing-a
- ako nema rezultata, sistem prelazi na /popular

Na taj način i novi korisnici dobijaju smislen početni prikaz preporučenih nekretnina.

2) Popularni listing-i (/popular)

Opis

Popular endpoint služi kao:

- fallback za Recommended
- zaseban prikaz popularnih / trending listing-a

Popularnost se računa kao kombinacija:

- pregleda u zadnjih 7 dana
- rezervacija u zadnjih 30 dana
- prosječne ocjene
- broja recenzija

Za razliku od /recommended, ovdje **nema personalizacije po korisniku**. Rangiranje je globalno i isto za sve korisnike.

2.1. Kandidati

Za popular listing-e sistem uzima:

- aktivne listing-e (`IsActive == true`)
 - newest 250 listing-a
 - za svakog kandidata cover sliku, ocjenu i broj recenzija
-

2.2. PopularScore formula

Za svaki listing računa se score prema pravilima:

- pregledi u 7 dana $\rightarrow + \min(15, \text{views7d} / 2)$
- rezervacije u 30 dana $\rightarrow + \min(30, \text{reservations} * 3)$
- rating $\rightarrow + \min(10, \text{AvgRating} * 2)$
- broj recenzija $\rightarrow + \min(5, \text{ReviewsCount} / 10)$

Nakon toga se listing-i sortiraju:

1. po score-u opadajuće
2. po CreatedAt opadajuće

i vraća se Top-N rezultat.

3) Entiteti koji učestvuju u preporukama

U trenutnoj implementaciji recommender koristi sljedeće entitete iz baze:

ListingView

Polja:

- UserId
- ListingId
- ViewedAt

Koristi se za:

- zadnje korisnikove preglede
- globalni broj pregleda u zadnjih 7 dana

Reservation

Polja:

- UserId
- PropertyId
- CreatedAt

Koristi se za:

- zadnje korisnikove rezervacije
- exclude listu
- globalni broj rezervacija u zadnjih 30 dana

Listing

Polja:

- Id
- Name
- CityId
- RentTypeId
- PricePerNight
- RoomsCount
- MaxGuests
- DistanceToCenterKm
- IsActive
- CreatedAt

Koristi se za:

- formiranje kandidata
- izgradnju historijskih profila
- osnovne podatke za scoring i prikaz

Review

Koristi se za:

- AvgRating
- ReviewsCount

PropertyImage

Koristi se za:

- dohvat naslovne slike (cover)
- fallback na prvu sliku po SortOrder

City i RentType

Koriste se za:

- prikaz naziva grada i tipa najma u rezultatu

5) Putanja do source code-a

- RentLoop.API/Controllers/ListingsController.cs

```
255 [HttpGet("recommended")]
256 [Authorize]
257 public async Task<IActionResult> Recommended([FromQuery] int take = 15)
258 {
259     take = Math.Clamp(take, 1, 50);
260
261     var userId = GetUserIdOrNull();
262     if (!userId.HasValue) return Unauthorized();
263
264     var recentViewedIds = await _db.ListingViews
265         .AsNoTracking()
266         .Where(v => v.UserId == userId.Value)
267         .OrderByDescending(v => v.ViewedAt)
268         .Select(v => v.ListingId)
269         .Distinct()
270         .Take(10)
271         .ToListAsync();
272
273     var recentReservedIds = await _db.Reservations
274         .AsNoTracking()
275         .Where(r => r.UserId == userId.Value)
276         .OrderByDescending(r => r.CreatedAt)
277         .Select(r => r.PropertyId)
278         .Distinct()
279         .Take(5)
280         .ToListAsync();
281
282     var exclude = recentReservedIds;
283
284     var fromViews = DateTime.UtcNow.AddDays(-7);
285     var fromRes = DateTime.UtcNow.AddDays(-30);
```

```

287     var views7d = await _db.ListingViews
288     .AsNoTracking()
289     .Where(v => v.ViewedAt >= fromViews)
290     .GroupBy(v => v.ListingId)
291     .Select(g => new { ListingId = g.Key, Cnt = g.Count() })
292     .ToDictionaryAsync(x => x.ListingId, x => x.Cnt);
293
294     var res30d = await _db.Reservations
295     .AsNoTracking()
296     .Where(r => r.CreatedAt >= fromRes)
297     .GroupBy(r => r.PropertyId)
298     .Select(g => new { ListingId = g.Key, Cnt = g.Count() })
299     .ToDictionaryAsync(x => x.ListingId, x => x.Cnt);
300
301     var historyIds = recentViewedIds.Concat(recentReservedIds).Distinct().ToList();
302
303     var historyListings = await _db.Listings
304     .AsNoTracking()
305     .Where(l => historyIds.Contains(l.Id))
306     .Select(l => new ListingHistoryProfile
307     {
308         Id = l.Id,
309         CityId = l.CityId,
310         RentTypeId = l.RentTypeId,
311         PricePerNight = l.PricePerNight,
312         RoomsCount = l.RoomsCount
313     })
314     .ToListAsync();
315
316     var baseQuery = _db.Listings
317     .AsNoTracking()
318     .Include(l => l.City)
319     .Include(l => l.RentType)

```

```

316     var baseQuery = _db.Listings
317     .AsNoTracking()
318     .Include(l => l.City)
319     .Include(l => l.RentType)
320     .Where(l => l.IsActive && !exclude.Contains(l.Id));
321
322     var candidates = await baseQuery
323     .OrderByDescending(l => l.CreatedAt)
324     .Take(250)
325     .Select(l => new ListingScoreItem
326     {
327         Id = l.Id,
328         Name = l.Name,
329         PricePerNight = l.PricePerNight,
330         City = l.City != null ? l.City.Name : "",
331         RentType = l.RentType != null ? l.RentType.Name : "",
332         CityId = l.CityId,
333         RentTypeId = l.RentTypeId,
334         RoomsCount = l.RoomsCount,
335         MaxGuests = l.MaxGuests,
336         DistanceToCenterKm = l.DistanceToCenterKm,
337         CreatedAt = l.CreatedAt,
338
339         CoverUrl = _db.PropertyImages
340         .Where(pi => pi.PropertyId == l.Id && pi.IsCover)
341         .Select(pi => pi.Url)
342         .FirstOrDefault()
343         ?? _db.PropertyImages
344         .Where(pi => pi.PropertyId == l.Id)
345         .OrderBy(pi => pi.SortOrder)
346         .Select(pi => pi.Url)
347         .FirstOrDefault(),
348
349         AvgRating = _db.Reviews
350         .Where(rv => rv.PropertyId == l.Id)

```

```

349         AvgRating = _db.Reviews
350         .Where(rv => rv.PropertyId == l.Id)
351         .Select(rv => (double?)rv.Rating)
352         .Average() ?? 0,
353
354         ReviewsCount = _db.Reviews.Count(rv => rv.PropertyId == l.Id)
355     })
356     .ToListAsync();
357
358     int Score(ListingScoreItem c)
359     {
360         int score = 0;
361
362         foreach (var h in historyListings)
363         {
364             if (c.CityId == h.CityId) score += 6;
365             if (c.RentTypeId == h.RentTypeId) score += 5;
366
367             var lower = h.PricePerNight * 0.8m;
368             var upper = h.PricePerNight * 1.2m;
369             if (c.PricePerNight >= lower && c.PricePerNight <= upper) score += 4;
370
371             if (c.RoomsCount == h.RoomsCount) score += 2;
372         }
373
374         if (views7d.TryGetValue(c.Id, out var vCnt)) score += Math.Min(10, vCnt / 3);
375         if (res30d.TryGetValue(c.Id, out var rCnt)) score += Math.Min(15, rCnt * 2);
376
377         score += (int)Math.Min(10, c.AvgRating * 2);
378         score += (int)Math.Min(5, c.ReviewsCount / 10);
379
380         return score;
381     }

```

```
383     var ranked = candidates
384     .Select(x => new
385     {
386         x.Id,
387         x.Name,
388         x.PricePerNight,
389         x.City,
390         x.RentType,
391         x.RoomsCount,
392         x.MaxGuests,
393         x.DistanceToCenterKm,
394         x.CoverUrl,
395         x.AvgRating,
396         x.ReviewsCount,
397         Score = Score(x),
398         x.CreatedAt
399     })
400     .OrderByDescending(x => x.Score)
401     .ThenByDescending(x => x.CreatedAt)
402     .Take(take)
403     .ToList();
404
405     if (ranked.Count == 0)
406         return await Popular(take);
407
408     return Ok(ranked);
409 }
```